Company A: Direct Booking and AI Enquiry Inbox

Technical Design Document

Item	Link
GitHub	https://github.com/Bored-Pawar/ISOCODE_Resort_booking_and_management_Platform
Live app	https://isocode-resort-booking-and-manageme.vercel.app/
Run and deploy guide	`deployment.md` in this ZIP (also in the repository root)
Date	9 November 2026

Demo logins. All demo accounts use the public demo password `Demo@12345`. They hold synthetic data only.

Role	Email	Sign in at	Sees
Guest	`asha@guest.example`	`/login`	Search, booking, My bookings
Reservations	`reservations@companya.example`	`/admin/login`	Rates, allotment, blackouts, promos, bookings, room enquiries
Sales	`sales@companya.example`	`/admin/login`	Wedding and offsite enquiries
GM	`gm@companya.example`	`/admin/login`	Everything, plus the Unsorted lane and complaint alerts

Test tools on the live app: `/dev/test` (simulated email and WhatsApp) and `/dev/outbox` (where approved replies and booking emails land). Nothing is charged, emailed or messaged.

---

1. Summary

Problem. Company A is a 76 key coffee estate resort in Chikkamagaluru that sells 68% of room nights through OTAs at 18 to 22% commission and only 12% direct, because its website is a brochure. Enquiries for rooms, weddings and offsites arrive by email, phone and the GM's WhatsApp, and rates live in Excel, so replies are slow and some events are lost.

What was built. One web app with two halves:

1. A direct booking site: rooms with photos, live availability and prices including GST, a three step booking flow (room, details, test payment) and a confirmation. The site sells only the allotment staff set, and cannot overbook.
2. A staff back office: daily rates, website allotment, blackout dates and promo codes, plus one enquiry inbox fed by the web form, simulated email and simulated WhatsApp. An AI router classifies each enquiry, extracts dates, guests and budget, assigns an owner and drafts a reply. Prices in drafts come only from the rate table. A human approves every reply.

Business case (from the brief) and how the build supports it.

Brief figure	Value
Current OTA commission	₹2.22 crore a year
Saving if 15% of OTA nights shift direct	₹33.3 lakh a year
Events lost to slow replies	₹72 lakh a year
Total opportunity	₹1.05 crore a year

Success target	How the build supports it	Measured today?
Direct share 12% to 20% in 12 months	Bookable direct site with live price including GST, promo codes, at most three steps, fast pages on slow networks (live median LCP under 2.5 s on all five measured pages, section 8)	No. Needs real traffic after launch
Median first reply under 30 minutes in working hours	Every enquiry is classified and drafted within seconds (R5 median model latency 1,588 ms), routed to an owner lane, and replied to in one approval. `first_reply_at` is stored on each enquiry	No. The Q8 metrics screen was cut (section 11). The data needed is stored
Zero website overbookings	Database constraint `sold + held <= allotment`, ordered row locks, atomic all or nothing holds, payment confirmed exactly once. Twenty parallel requests for the last unit produce exactly one winner (section 6b)	Yes, by the PostgreSQL integration suite (176 passing at its last full run; the full run started today did not complete, section 10)

---

2. Scope

Brief item	Status
Room pages with photos, availability and price with GST	Done
Booking flow of at most 3 steps, test payment, confirmation	Done. Carts of 1 to 4 rooms
Admin: daily rates, website allotment, blackout dates, promo codes	Done
Enquiry inbox: web form, forwarded email, WhatsApp (simulated)	Done
AI router: classify, extract, assign owner, draft reply	Done, with failure handling
Roles: guest, reservations, plus GM view (Unsorted lane, complaint alerts)	Done. A sales role was added for weddings and offsites
Optional: structured wedding and event form	Folded into the enquiry form (event fields appear for Wedding or Offsite)
Optional: daily booking export	Not built (section 11)

Roles.

Role	Can do
Guest	Search, build a stay, sign up inline, hold, pay (test), see and cancel own holds, send an enquiry
Reservations	Rates, allotment, blackouts, promos, bookings, room and other enquiries
Sales	Wedding and offsite enquiries; can ask reservations and the GM to look at an enquiry
GM	Everything above, plus the Unsorted lane, complaint alerts, failed AI retry

3. Assumptions

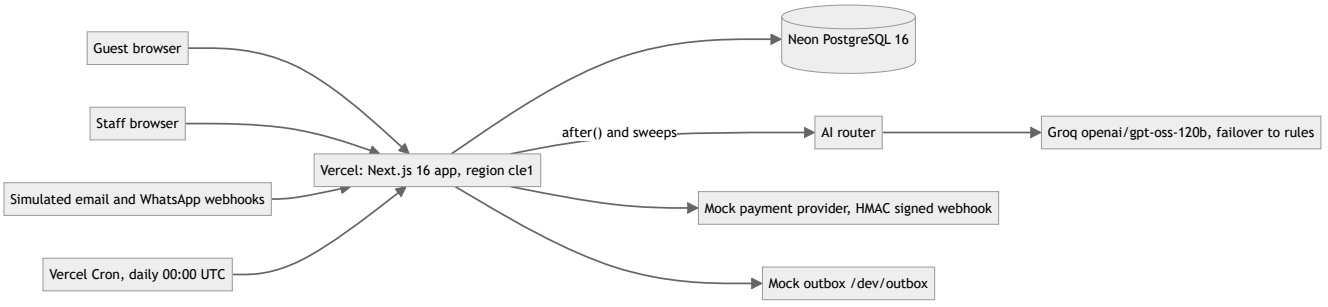
The brief's own assumptions are A1 to A10. A11 onward are decisions made during the build. Values are taken from the code.

ID	Assumption	Source
A1	Inventory is sold by room type, not physical room number. The reservation system assigns rooms	brief
A2	GST slab is chosen per night on the room value after discount. Slabs are stored in a table	brief, `tax_slabs`
A3	Hold time is 10 minutes (`HOLD_MINUTES`, default 10)	brief, `src/modules/booking/transaction.ts`
A4	Superseded: the brief assumed one room per booking. We allow 1 to 4 rooms per booking. More than 4 rooms, or not enough stock, goes to a group enquiry	`booking_rooms.line_no` CHECK 1 to 4
A5	Guests sign in with email and password, created inline in booking step 2	brief
A6	Room type names, photos and rates are invented	brief
A7	The brand is shown as "Company A" and is configurable in `src/lib/site.ts`	brief
A8	Photos are free stock or generated placeholders	brief
A9	Room types: Plantation Room 40, Estate Suite 24, Garden Villa 8, Pool Villa 4 (76 keys)	brief, seed
A10	Online check-in no later than today (IST) plus 3 calendar months, month end clamps. Later dates go to an enquiry	`src/lib/booking-window.ts`
A11	GST: 5% for a night up to ₹7,500, 18% above, bounds inclusive, effective 2025-09-22. To be checked by the client's accountant before go live	`src/db/seed/data.ts`
A12	Website allotment seeded at about 40% of units: 16, 10, 3 and 2. The rest is agency and walk-in stock	seed
A13	Rate plans EP (room only), CP (+₹800, breakfast), MAP (+₹2,200, non refundable). Weekend = Friday and Saturday nights. Season multipliers: December and January 1.3, June to August 0.8, other months 0.85 to 1.1	seed
A14	Occupancy per room: at least 1 adult, adults up to the type maximum, adults plus children up to max occupancy (3, 4, 5, 6)	`src/lib/occupancy.ts`
A15	Online search allows up to 14 nights and 24 guests that fit in 4 rooms	`src/app/(guest)/_lib/search.ts`
A16	Payment cannot start with under 2 minutes of hold left. "Refresh hold" cancels and takes a fresh hold	pay panel, booking core
A17	A payment that arrives after the hold expired is recorded as REFUND_PENDING. The booking stays EXPIRED and no stock is taken again	`docs/01-booking-core.md`
A18	Blackouts block new holds only. Existing bookings stand	booking core
A19	Allotment cuts apply per date: dates that would drop below sold plus held are refused and reported, other dates apply	admin A2
A20	Promo value is a percentage (1 to 100) or a flat amount in whole rupees. One promo per booking. Validity applies to stay nights	admin A4
A21	Rate edits: whole rupees 1 to 5,00,000, dates from today to today plus 179 (180 day horizon)	admin A1

ID	Assumption	Source
A22	Hold expiry is lazy (before every availability read, hold and confirm) plus one daily cron as a safety net	`vercel.json`
A23	AI confidence below 0.70, category "other", or any AI failure sends the enquiry to the Unsorted lane with no owner	`src/modules/ai-router/routing.ts`
A24	The rules fallback classifier caps confidence at 0.5 (0.3 for "other"), so its results always land in Unsorted	`src/modules/ai-router/fallback.ts`
A25	AI price estimates use EP room only plans, up to 4 rooms, and are indicative. No room is held	AI router
A26	An omitted year means the next date that is not in the past. Omitted children are disclosed as "assumes no children"	AI router grounding
A27	Working hours for the first reply target are not defined in the brief or the code. The app stores `first_reply_at`, so the median can be computed once hours are agreed	none
A28	All data is synthetic: guests, enquiries, phone numbers and emails. Free tier AI providers receive synthetic data only	seed, `docs/04-ai-router.md`
A29	Payment, email and WhatsApp are mocks with real provider payload shapes (Postmark, Meta Cloud API). A shared secret header stands in for provider signatures	section 6d
A30	The first account of a role (by creation time) is the default owner, and the first GM receives alerts	`docs/02-architecture.md`
A31	Passport details are never taken online. The confirmation tells foreign guests to bring a passport for Form C at the desk	confirmation page
A32	Demo accounts share one public password. Accepted for a judged demo with synthetic data	`deployment.md`

4. Architecture

Style. A modular monolith: one Next.js app, one PostgreSQL database, one deploy. One developer had about 72 hours; separate services would add network failure modes and deploy work with no benefit at this size.



Choice	Why	Rejected alternative
Next.js 16 App Router, React 19, TypeScript, server actions	One codebase for guest pages, staff pages and APIs. Server components send little JavaScript, which matters on hill networks	Separate SPA and API: more JavaScript and two deploys

Choice	Why	Rejected alternative
Vercel, functions pinned to `cle1`	Zero config deploys from `main`, preview URLs, cron. `cle1` sits in the same AWS region as the Neon database (`us-east-2`) so each query is a few milliseconds	A VM: more operations work for one developer
PostgreSQL on Neon, Drizzle ORM, raw SQL for locks	Overbooking is prevented by row locks and a CHECK constraint, which needs a real relational database with transactions	A document store: no multi row transactions of the kind needed
Auth.js v5, credentials, JWT sessions	Separate guest and staff sign in. Roles carried in the session	Third party auth: extra setup for reviewers
Role table in `src/lib/auth/permissions.ts`	One role by capability table drives navigation, page guards and server action guards	Checks scattered per page
Zod on every input and every AI output	One validation layer for forms, webhooks and model answers	Trusting the model's JSON
Vercel AI SDK 7, `generateText` with structured output	One transport for Groq, Gemini, Anthropic or a local model	Provider specific SDKs
`after()` for AI work, plus sweeps	The enquiry is saved first and the sender gets a reference at once. AI runs after the response	A queue service: more moving parts
Daily Vercel Cron `/api/cron/expire-holds`	Safety net for hold expiry and stuck AI work. The Hobby plan allows daily crons only, so expiry is also lazy on every read	Minute crons: not available on the plan

Modules. Each module exposes one `index.ts`. Others may call only that file.

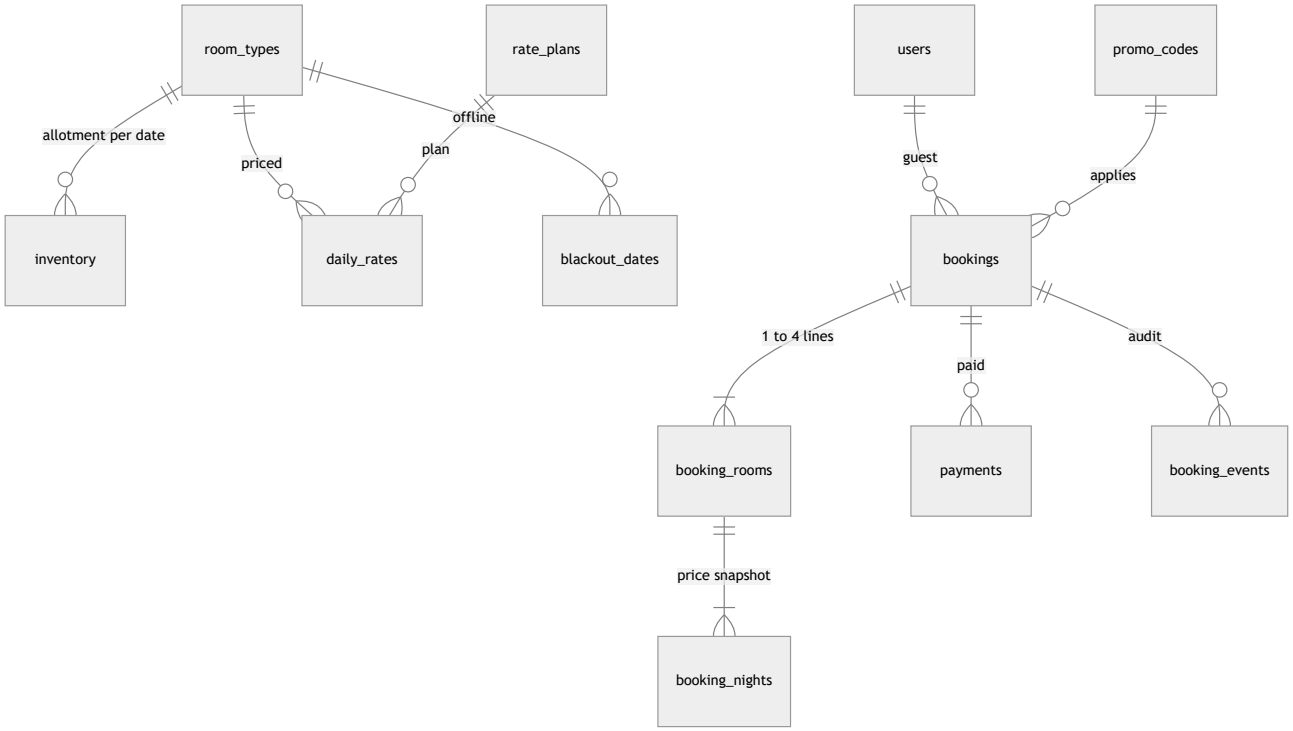
Module	Owns
`catalog`	Room types, photos, rate plans (read only)
`pricing`	Daily rates, promos, GST slabs, `quote()` and `quoteHash`. The only place prices are computed
`inventory`	Allotment, held and sold counters, blackouts, availability
`booking`	Booking state machine. Owns every transaction that spans modules
`payments`	Payment provider interface, mock provider, signed webhook handler
`enquiries`	Intake, inbox queries, claims, send
`ai-router`	Classification, PII redaction, grounding, routing, quote selection, draft, guard, recovery
`ai-provider`	Provider selection and one structured transport. No database access
`admin`	Facade for the rates, allotment, blackout and promo editors

Request path. Page or server action, Zod parse, role check (`requireRole`, `requireGuestOwner` or `requirePermission`), module `index.ts`, then SQL. Business failures return `{ ok: false, code }`, never throw. Money is integer paise. Dates are IST `DATE`.

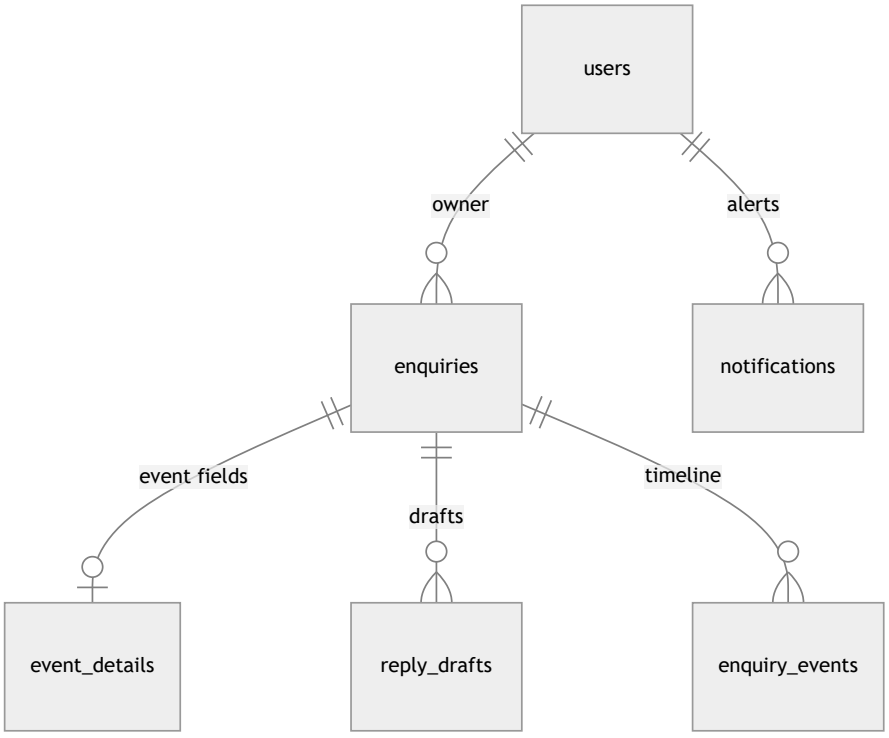
---

5. Data model

Booking and inventory side



Enquiry side



Also: `room_photos` (per room type), `tax_slabs` (GST bands, rate frozen into each booking night) and `export_runs` (unused).

20 tables. Money columns end in `_paise`. The tables that matter most:

Table	Key columns and constraints
`inventory`	Primary key (room type, date). `allotment`, `sold`, `held` each `>= 0`. **CHECK `sold + held <= allotment`** (`inventory_capacity_check`)
`bookings`	`ref` unique (`CA-` plus 5 characters). Status CHECK in HELD, CONFIRMED, EXPIRED, CANCELLED. CHECK `check_out > check_in`. `hold_expires_at`, `quote_hash`, subtotal, discount, GST and total in paise
`booking_rooms`	`line_no` CHECK 1 to 4, unique per booking. Adults `>= 1`, children `>= 0`
`booking_nights`	Primary key (booking room, date). Rate, discount, GST rate and GST frozen at hold time, so later rate edits never change a booking
`payments`	`provider_payment_id` unique: a duplicate webhook cannot confirm twice. Status SUCCESS, FAILED, REFUND_PENDING
`booking_events`	Every state change, written in the same transaction as the change
`daily_rates`	Primary key (room type, rate plan, date). Price `> 0`
`tax_slabs`	Lower and upper bound in paise, GST in basis points, `effective_from`
`promo_codes`	Code unique, kind pct or flat, value `> 0`, `used_count`, `max_uses`, `min_nights`, validity dates
`blackout_dates`	Room type (null means all rooms), CHECK `end_date >= start_date`, required reason
`enquiries`	Channel CHECK web, email, whatsapp, event_form. Category, confidence, extracted JSON, `ai_status`, `ai_attempts`, lane sorted or unsorted, owner, `is_complaint`, `first_reply_at`
`reply_drafts`	Body, quote JSON (from pricing), provider, model, prompt version, `approved_by`, `sent_at`
`enquiry_events`, `notifications`, `event_details`	Timeline (classified, rerouted, edited, sent, closed), GM alerts, wedding and offsite fields
`users`	Email unique, role CHECK guest, reservations, sales, gm. `consent_at` and `consent_version` (DPDP)
`export_runs`	Reserved for the daily export. Not used (section 11)

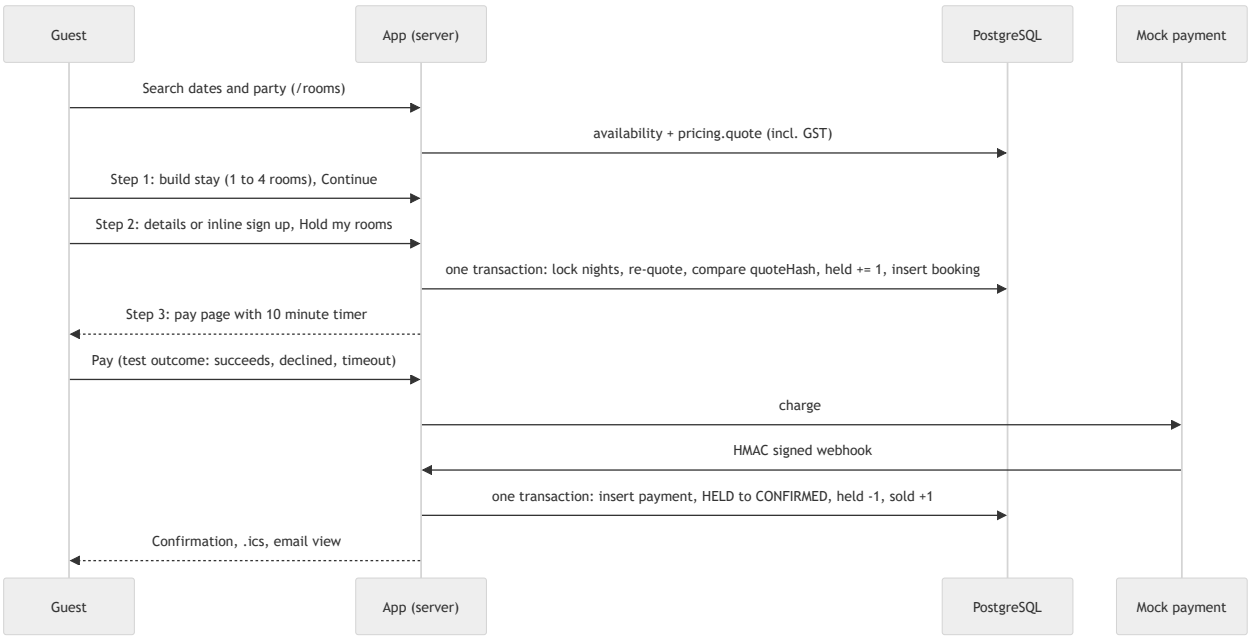
What prevents overbooking, layer by layer.

1. The CHECK constraint on `inventory` makes an oversold row impossible to commit, whatever the code does.
2. Holds lock every inventory row they touch with one `SELECT ... FOR UPDATE` ordered by (room type, date), across all room types of the cart. Same order everywhere means queues, not deadlocks.
3. Every state change is one conditional update, for example `UPDATE bookings SET status = 'CONFIRMED' WHERE id = $1 AND status = 'HELD' AND hold_expires_at > now()`.
4. The unique `provider_payment_id` stops a duplicate payment webhook.
5. Deadlock or serialization errors (`40P01`, `40001`) retry the whole transaction once, then answer SOLD_OUT.

---

6. Main flows

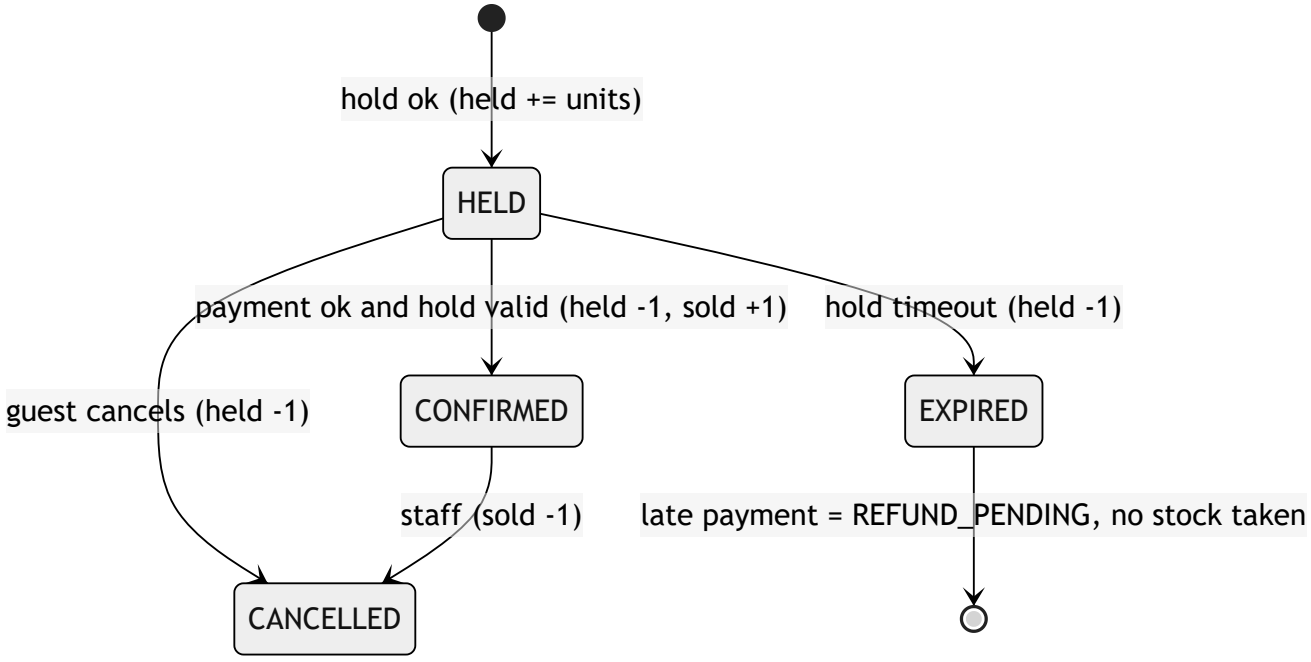
6a. Search, book, pay, confirm



1. Search on `/` or `/rooms`. Each room card shows stock left and the total including GST, computed on the server.
2. Step 1 (`/rooms`, room detail at `/rooms/garden-villa`): add up to 4 rooms. The cart lives in the URL. The server re-quotes on every change.
3. Step 2 (`/book/details`): sign in, or sign up inline with DPDP consent. If the price changed since the quote, the guest sees old and new totals and continues at the new price.
4. Step 3 (`/book/pay/CA-XXXXX`): a 10 minute hold timer. Test outcomes: payment succeeds, card declined (hold kept, retry allowed), bank times out (Check status or Retry).
5. Confirmation (`/booking/CA-XXXXX`) with per room GST breakdown, calendar file and a rendered email. My bookings (`/account`) shows upcoming, past and not completed stays, with Resume payment and Cancel for a live hold.

6b. Allotment control (the hard part)

Rule: the website sells only what staff allot, and never more.



Acquire hold, one transaction.

1. Per guest advisory lock, so one guest's holds run one at a time. A repeat request for the same stay returns the existing hold (idempotent).
2. Release this stay's expired holds first (scoped lazy expiry).
3. Lock all nights of all room types in the cart, in (room type, date) order, in one statement.
4. If any night is missing, short of stock or blacked out, reject the whole cart. Nothing is taken (all or nothing).
5. Re-price inside the transaction and compare the `quoteHash` the guest saw. A mismatch returns PRICE_CHANGED with the new quote and takes no stock.
6. Add to `held`, insert the booking, room lines, frozen nightly prices and an event.

Expiry. `UPDATE bookings SET status = 'EXPIRED' WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED)`, then release `held` in the same transaction. Two expiry workers cannot release the same hold twice.

Payment exactly once. The payment row has a unique provider id. Confirmation is one conditional update from HELD. A duplicate webhook is a no op. A payment after expiry becomes REFUND_PENDING and the booking stays EXPIRED. A wrong amount is refused.

Admin cannot cut below what is committed. One conditional update: `... WHERE sold + held <= new_allotment AND new_allotment <= total_units`. Refused dates are shown with the reason ("At least N, already sold or held").

Tests that prove it (PostgreSQL integration suite; 176 of 176 passed at its last full run, see section 10):

#	Requirement	Test (file: name)
1	20 parallel holds on the last unit: exactly 1 wins	`booking.test.ts`: "mandatory 1: twenty simultaneous last-unit holds produce exactly one winner"
2	Multi night hold with one full night rolls back completely	`booking.test.ts`: "mandatory 2: one unavailable night rolls back the entire hold"
3	Two expiry callers release stock once	`inventory.test.ts`: "concurrent expiry callers release stock and write the transition once (mandatory test 3)"

#	Requirement	Test (file: name)
4	Duplicate payment webhook confirms once	`booking.test.ts`: "mandatory 4: parallel duplicate notifications confirm and consume the promo exactly once"
5	Late payment: REFUND_PENDING, booking stays EXPIRED, stock unchanged	`booking.test.ts`: "mandatory 5: repeated late payment records one refund and never reacquires another guest's inventory"
6	Allotment cut below committed is refused	`admin-allotment.test.ts`: "mandatory 6 via A2: a cut below sold + held is rejected for those dates only, the rest apply"
7	Hold on a blackout date is refused	`booking.test.ts`: "mandatory 7: a blackout rejects a hold without acquiring inventory"
8	GST slab correct on both sides of the boundary	`pricing.test.ts`: "reads both sides of the nightly GST boundary and reprices identically in a shared transaction"
9	Price changed between quote and hold	`booking.test.ts`: "mandatory 9: changed quote returns PRICE_CHANGED and rolls back even initial lazy expiry"
B10	Multi room carts	`multiroom.test.ts`: reverse order two type races with a deadlock counter check, one unavailable type leaves the other untouched, blackout rejects the whole cart
Idempotency	Same guest, same cart	`hold-idempotency.test.ts`: five parallel identical requests create one booking

6c. Admin: rates, allotment, blackouts, promos

Screen	What staff do	Rules
Rates `/admin/rates`	30 day grid per room type and plan. Edit one cell or bulk edit (rooms, plans, dates, weekdays) with a preview ("Updates N prices across M rooms")	Whole rupees 1 to 5,00,000. Today to today plus 179. Existing bookings keep their frozen nightly price. Open carts get PRICE_CHANGED
Allotment `/admin/allotment`	Grid of allotment with sold and held. Single or bulk edit	Cannot go below sold plus held, or above total units. Refused dates shown in red with the reason
Blackouts (panel on Allotment)	Take a room type or all rooms offline for a date range with a reason, or bring it back	Lists live bookings that overlap first. Blocks new holds only
Promos `/admin/promos`	List (Active, Off, Ended, Used up), create, switch off in two steps	Codes uppercase and unique. A switched off code returns PROMO_INVALID. Confirmed bookings keep their discount

Admin writes lock inventory rows in the same (room type, date) order as holds, so an edit and a guest hold queue instead of deadlocking. A test races an allotment cut against ten holds and the CHECK still holds.

6d. Enquiry intake into one inbox

1. **Web form** `/enquire`: Zod validation, honeypot, consent, optional event fields for Wedding or Offsite.
2. **Email** `/api/inbound/email`: Postmark shaped JSON.
3. **WhatsApp** `/api/inbound/whatsapp`: Meta Cloud API shaped JSON.
4. The webhooks need the `x-inbound-secret` header (constant time compare, fail closed), cap the body at 256 KB, and are idempotent on the provider message id.
5. `/dev/test` posts the same shapes through the same code, for reviewers.

All three call one function, `enquiries.ingestFrom()`. The row is saved with `ai_status = 'pending'` before any AI call, so a slow or failed model never loses an enquiry.

---

7. The AI part

Provider setup

Item	Value
Live provider	Groq, model `openai/gpt-oss-120b` (`AI_PROVIDER=groq`, `AI_MODEL`)
Other providers	`google`, `anthropic`, `local` (OpenAI compatible, for example Ollama at `LOCAL_AI_BASE_URL`), `rules` (no key, no network)
Selection	`src/modules/ai-provider/factory.ts`. Missing key or model means `NOT_CONFIGURED`, which falls back to rules
Keys	`GROQ_API_KEY_1` to `GROQ_API_KEY_4`, tried in order. A key that returns 429, 401 or 403 cools down in memory for Retry-After, else 60 s, and the next key is tried at once
Timeout	10 s per attempt (`ATTEMPT_TIMEOUT_MS`), at most one retry for transient errors, a 429 retried only if Retry-After is 1 s or less. Worst case about 21 s
Output	`maxOutputTokens: 1800`. Groq is called with a non strict JSON schema because strict mode rejected optional fields with HTTP 400
Calls per enquiry	One structured call returns classification, extraction and draft prose together

What it does



1. **Classify** into room, wedding, offsite, complaint or other, with a confidence.
2. **Extract** dates, adults and children, budget, room preference, language and a one line summary.
3. **Ground**: the server re-reads the original message and checks the extracted dates and counts. Anything conflicting or unsupported gets no price.
4. **Route**: room to reservations, wedding and offsite to sales, complaint to the GM.
5. **Draft**: the model writes prose with a `{{QUOTE}}` placeholder. The server fills it with a quote from `pricing.quote()` over the rate table, only when dates and guests are both grounded.

When it is wrong or unsure

Situation	What happens
Confidence below 0.70, or category "other"	Unsorted lane, no owner. Only the GM sees this lane
Provider error, timeout, rate limit, invalid JSON	Rules classifier, `ai_status = 'fallback'`, Unsorted lane
Wrong but confident category	One click reroute on the enquiry. Logged on the timeline

Situation	What happens
Any complaint	Detected from the model category, the model's complaint flag, or an English, Hinglish and Devanagari keyword check on the original text, whatever the confidence. The GM always gets an alert. Mixed messages keep their owner and still alert the GM
Model writes a price, discount, promise or link	The price guard rejects the prose and a safe template is used. Classification is kept
Dates or guests not clear	No price. The draft asks the guest
Worker crash mid processing	Row is reclaimed after 2 minutes. After 3 attempts it is marked failed and goes to the GM with a Retry button
Sending	Never automatic. Every reply needs a staff click and a confirm dialog

Quote integrity (QA-01, QA-02 and the failed rebuild risk: fixed on a branch, pending deployment)

Production QA found that staff edits could leave a wrong price sendable. A date only edit on a Garden Villa enquiry re-quoted the cheapest room (Plantation Room), and a Room only edit left the old Garden Villa price approvable; it was actually sent to the mock outbox. The fix:

1. **Three room states.** The room a quote uses is `message` (staff never touched it, the guest's own words decide), `staff` (staff typed a replacement) or `cleared` (staff removed it, so the draft asks which room). A date or guest edit no longer drops the guest's named villa. A typed room that matches no room type asks instead of quoting the cheapest room.
2. **Every quote affecting edit rebuilds.** Dates, guests and room all re-price the unsent draft.
3. **Facts recorded with each draft.** `extracted.quoteBasis` (existing JSON column, no migration) stores the fingerprint of the facts, the draft id and a key of its quote. The edit marks it `rebuilding` in the same transaction; a rebuild commits only if it is still the latest; a failure marks it `failed`.
4. **Approval checks facts under the enquiry lock.** Send recomputes the fingerprint from the saved facts inside the lock and refuses QUOTE_REBUILDING, QUOTE_FAILED or QUOTE_STALE. The older STALE_DRAFT and ALREADY_SENT checks still run first.
5. **The panel shows it.** "Quote being rebuilt" or "Quote out of date" with Retry, and Approve disabled. Each draft shows the dates, party and room it was built on.
6. **Legacy rule.** A draft with no recorded basis stays approvable only if no quote affecting field was ever edited. Drafts rebuilt by the old code after an edit must be rebuilt first.

Status: fixed and tested on branch `claude/s3-tdd-pack` (752 unit tests, a new 16 test PostgreSQL quote suite and the 18 test inbox suite pass). **Not yet deployed**: the live app still runs the previous code until this branch is merged (section 12).

Evaluation

30 case evaluation (R5, 30 labelled synthetic seed enquiries, prompt `enquiry-v2-audit`).

Metric	Groq `gpt-oss-20b`	Groq `gpt-oss-120b` (now live)	Rules only
Valid structured answer	21/30 (70%)	29/30	n/a

Metric	Groq `gpt-oss-20b`	Groq `gpt-oss-120b` (now live)	Rules only
Correct category	20/30 model answers; 29/30 end to end after fallback	28/30	30/30 (tuned to this seed, not a general claim)
Fallback to rules	9/30	1/30	30/30
Unsorted lane	13/30 (43%)	6/30	30/30
Complaint recall, GM alerted (n = 5)	5/5	5/5	5/5
GM alerts on non complaints	0	not recorded	0
Draft template fallback	14/30 (47%)	7/30	30/30
Median latency	1,588 ms (p95 2,346 ms)	1,716 ms	n/a

What it taught:

1. Groq's strict JSON mode returned HTTP 400 on every call because optional fields were not listed as required. Fixed by sending a non strict schema; Zod still validates every answer.
2. `gpt-oss-20b` sometimes answered with the JSON schema itself, the likely cause of most of its 9 fallbacks. Moving to `gpt-oss-120b` cut fallbacks to 1. This is a mitigation, not a cure: one 120b call (ENQ-4397) still failed and went to Unsorted, as designed.
3. Complaint recall stayed at 5/5 in every run because the keyword detector on the original text does not depend on the model.
4. The failures all went to Unsorted for a human. No wrong price was produced by the model, because the model never produces prices.

Live production smoke (production QA, 9 November 2026, five synthetic enquiries).

Case	Result
Named villa (WhatsApp)	Room, 96%, Reservations, Garden Villa ₹38,350 including ₹5,850 GST from the rate table
Wedding (web form)	Wedding, 96%, Sales, budget extracted, no invented price
Offsite (email)	Offsite, 96%, Sales, unpriced draft
Complaint (email)	Complaint, 96%, GM owner and a GM alert
Ambiguous (WhatsApp)	Other, 60%, Unsorted, no owner. The GM rerouted it in one click

Five of five landed in the expected lane. This is a small sample and not an accuracy claim.

Privacy

Sent to the provider	Never sent
The message text after redaction, and today's date (IST)	Sender name, email and phone fields, consent metadata, message ids
	Emails, mobile numbers, landlines, PIN code address lines, sign off names, common ID numbers in the text (replaced before the call)

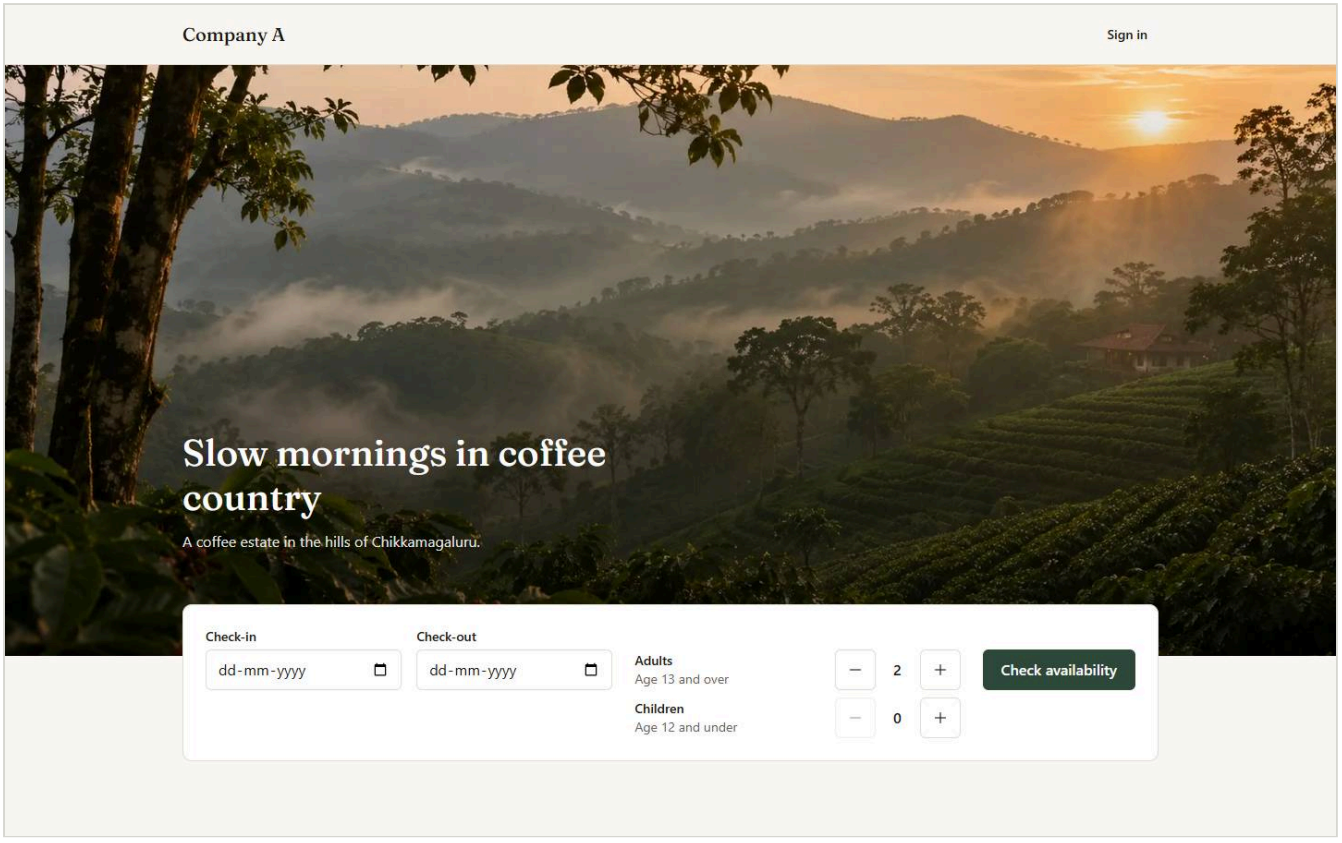
The original text stays on our server for grounding and routing. Redaction is pattern based minimisation, not full entity recognition: an unusual name or address can slip through, so only synthetic data is used with free tier providers.

---

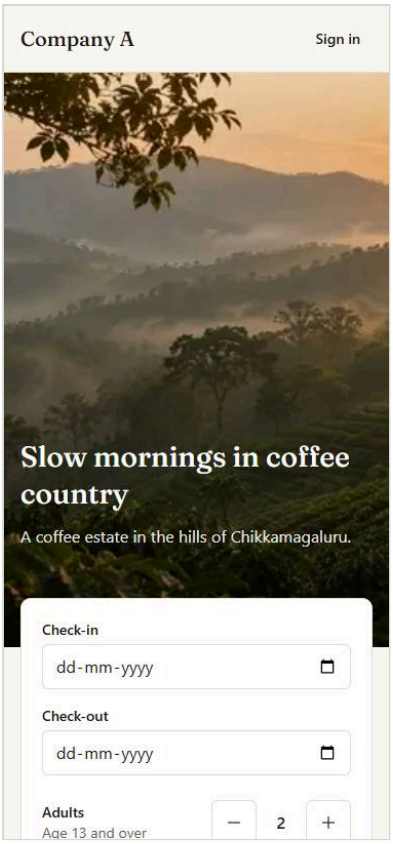
8. UI and design decisions

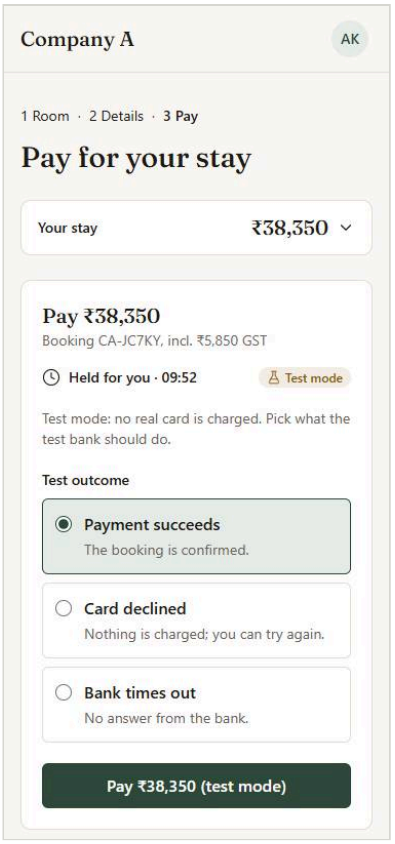
The brief asks for minimal design: white space, a restrained palette, clear type, large photos, and a booking flow of at most three steps. The design system ("Quiet estate") uses one accent colour (estate green `#2F4A3A`) for the primary action only, warm paper background `#FAF8F4`, Fraunces for headings and guest prices, system sans for everything else, borders instead of shadows, 44 px tap targets, and status colours always paired with a word or icon.

Guest screens

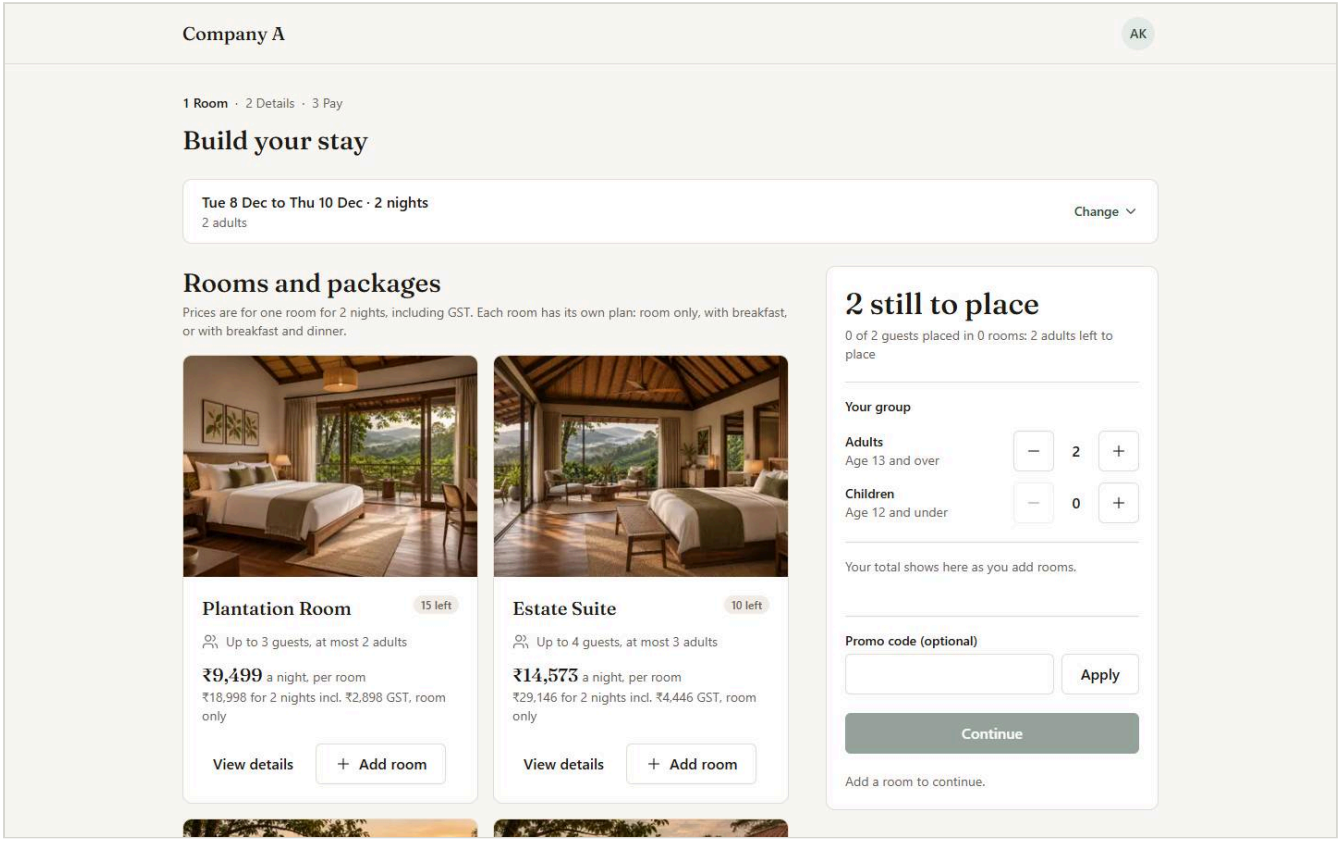


Home. A full width photo of the estate with the search card on top. Dates and party are the only inputs. Three short story blocks and room highlights below.

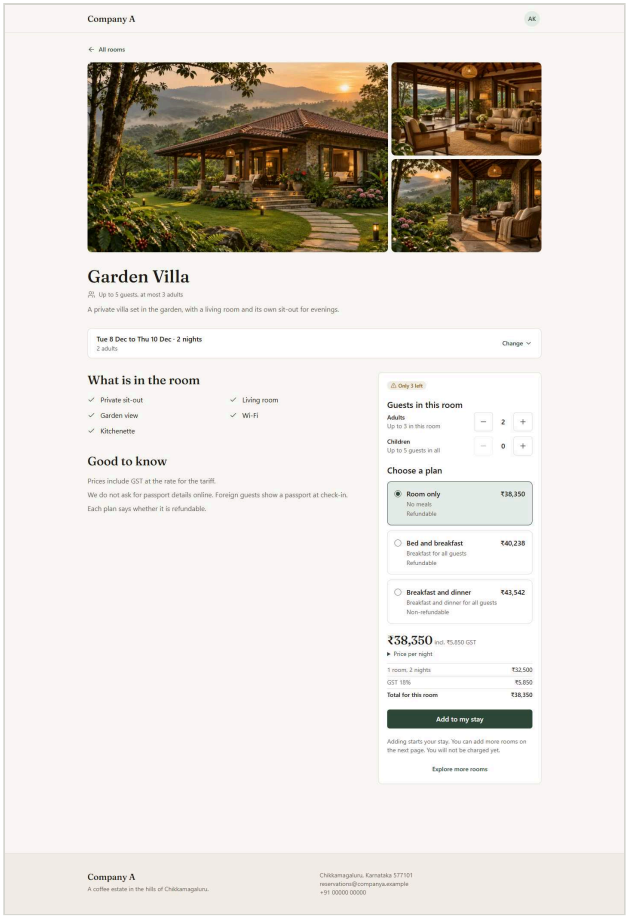




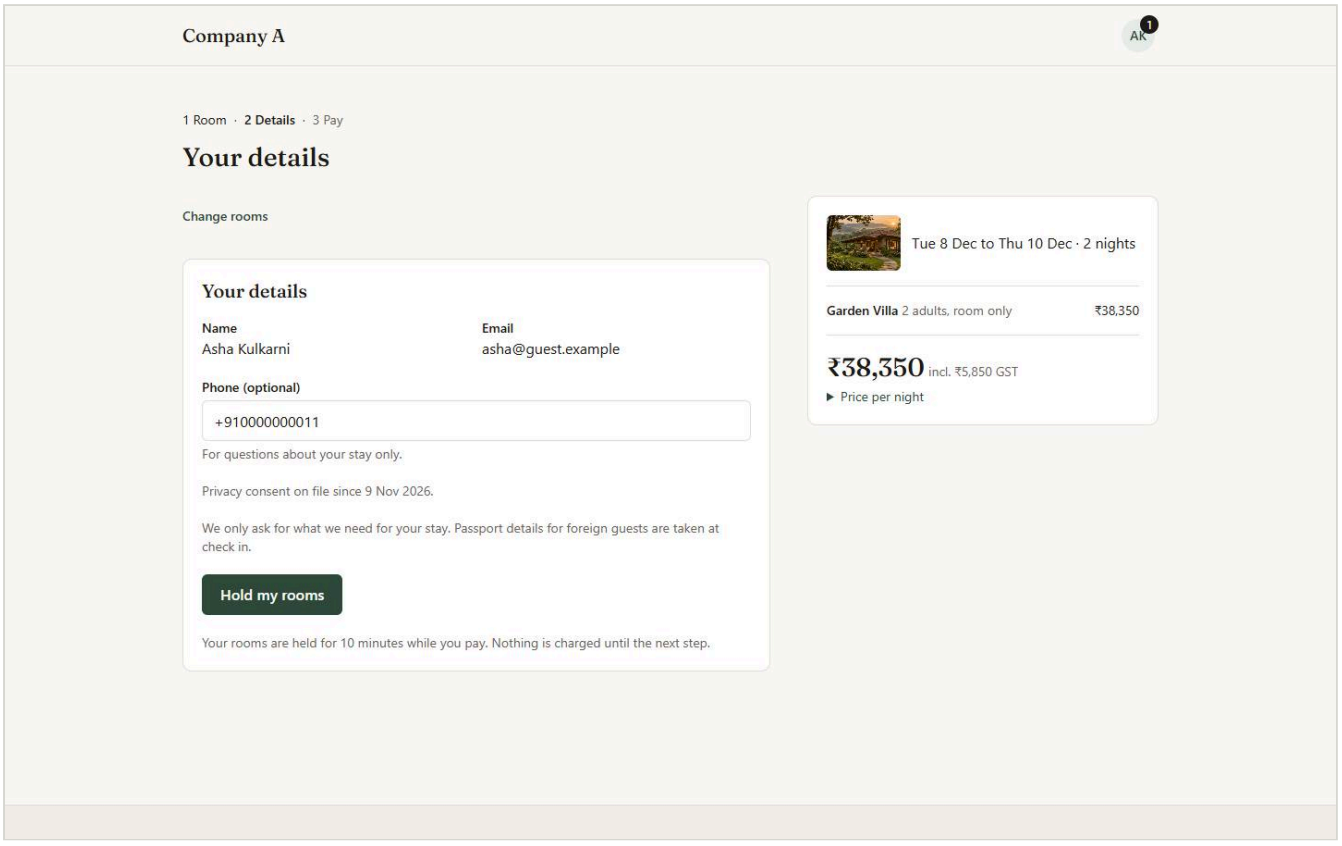
Mobile. The same order on a 375 px phone, one column, thumb sized controls. The pay step keeps the total, timer and outcomes on one screen.



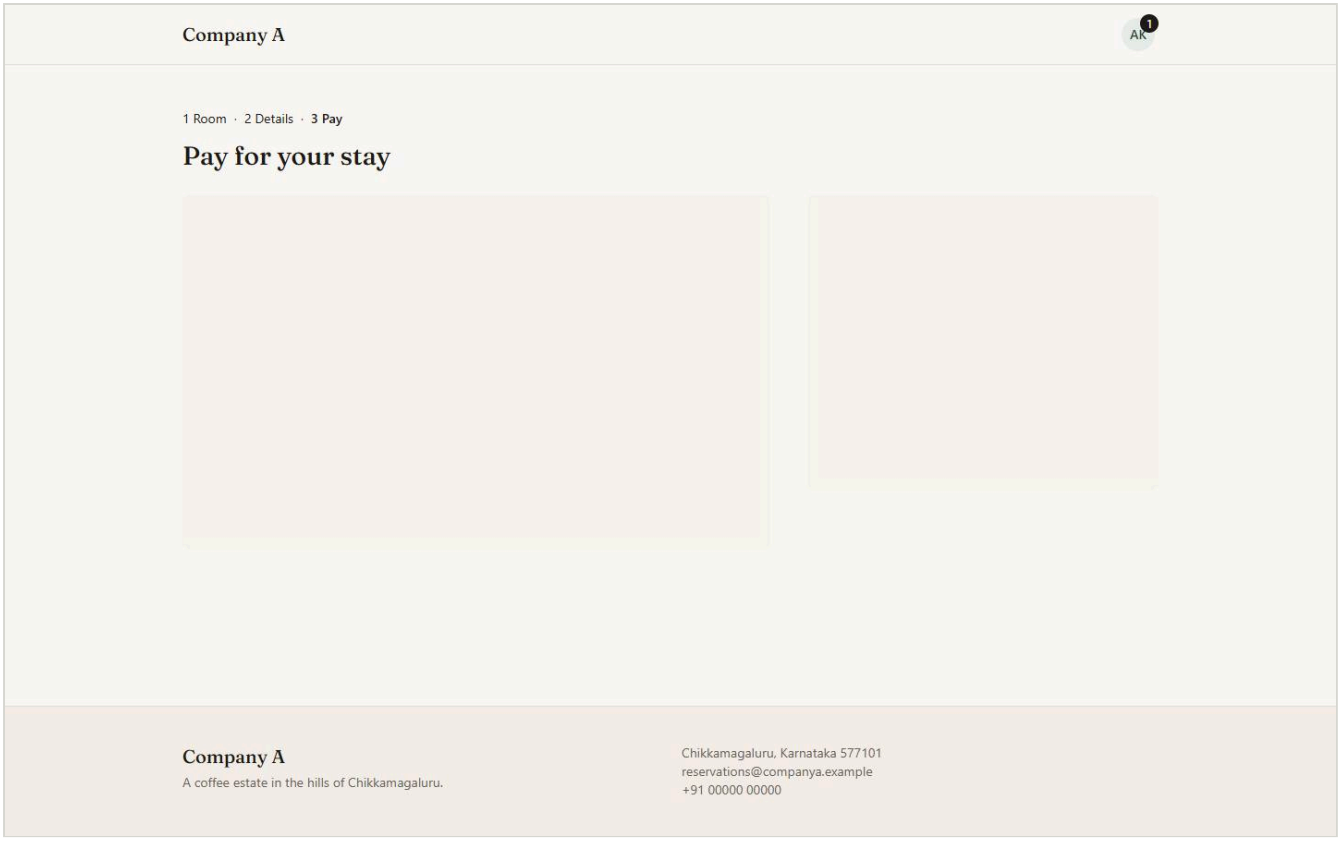
Rooms. One card per room type with a large photo, stock left and the price including GST, always visible. The stay summary stays in view and says how many guests still need a room.



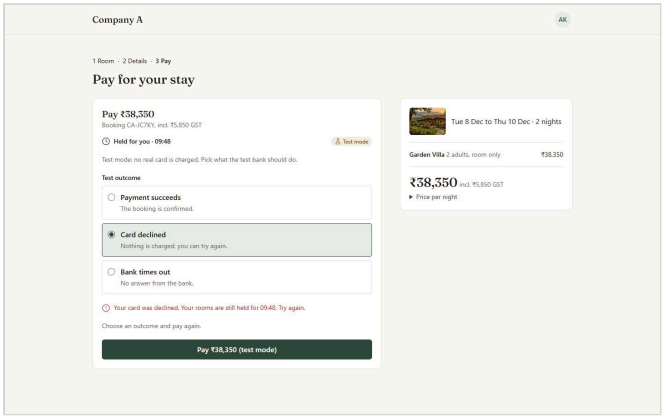
Room detail. One large photo and two smaller ones, amenities, plans as radio cards, and the total with a GST line per slab.

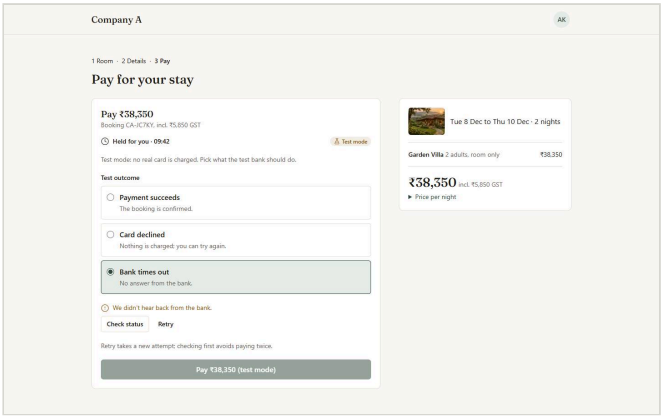


Step 2. A step indicator (1 Room, 2 Details, 3 Pay). We ask only for what the stay needs; a note says passport details are taken at check in.

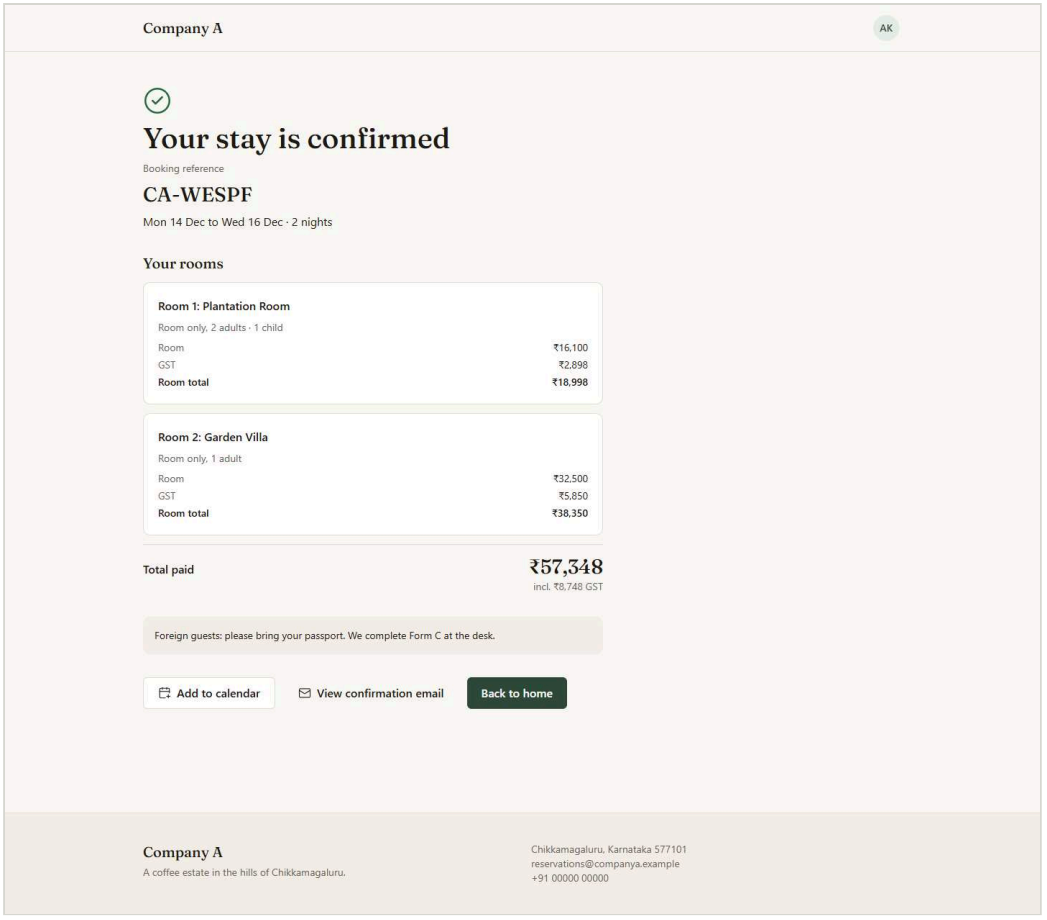


Step 3. "Test mode" is labelled. A hold timer warns under 2 minutes. Three test outcomes as radio cards.

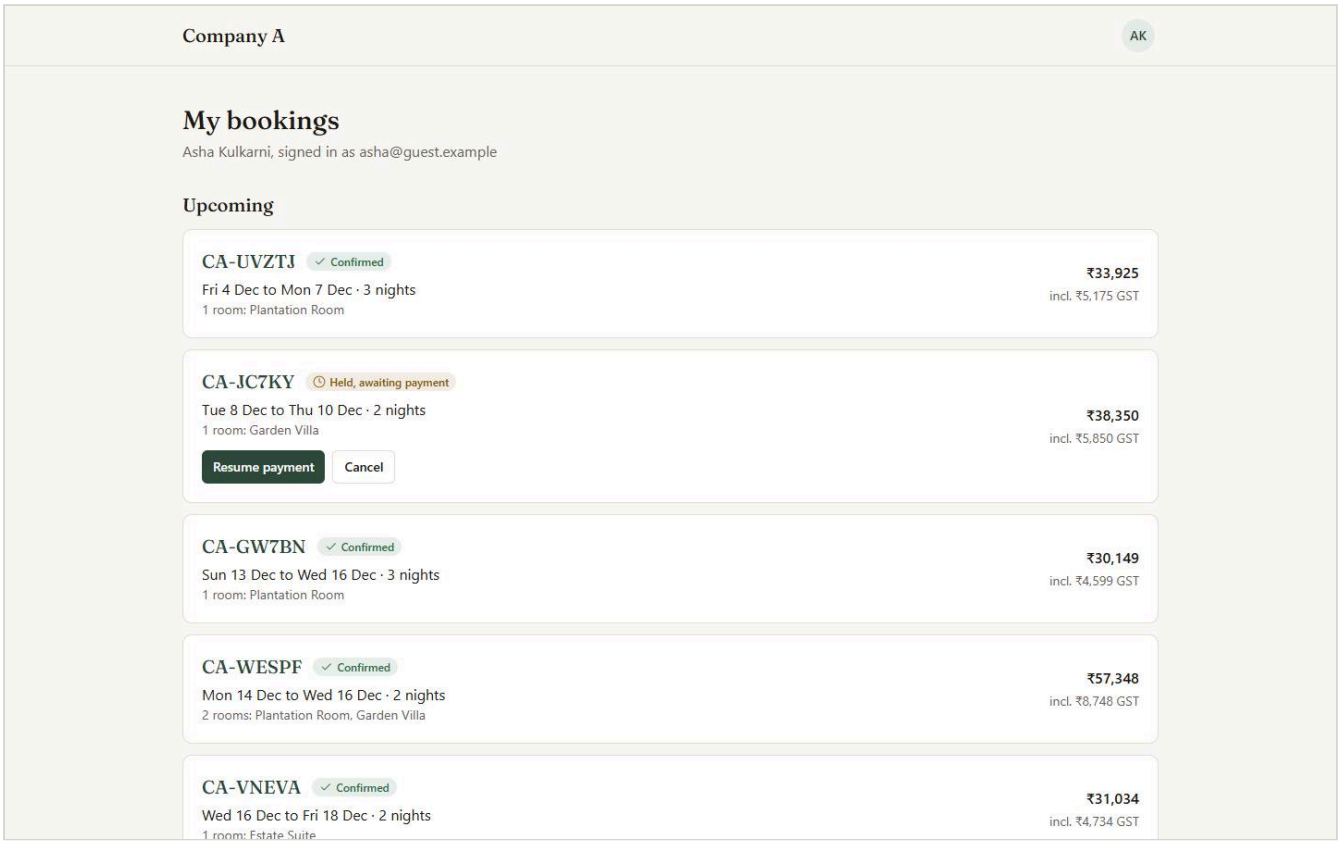




Payment failure. Declined keeps the hold and allows a retry. Timeout offers Check status first, to avoid paying twice.

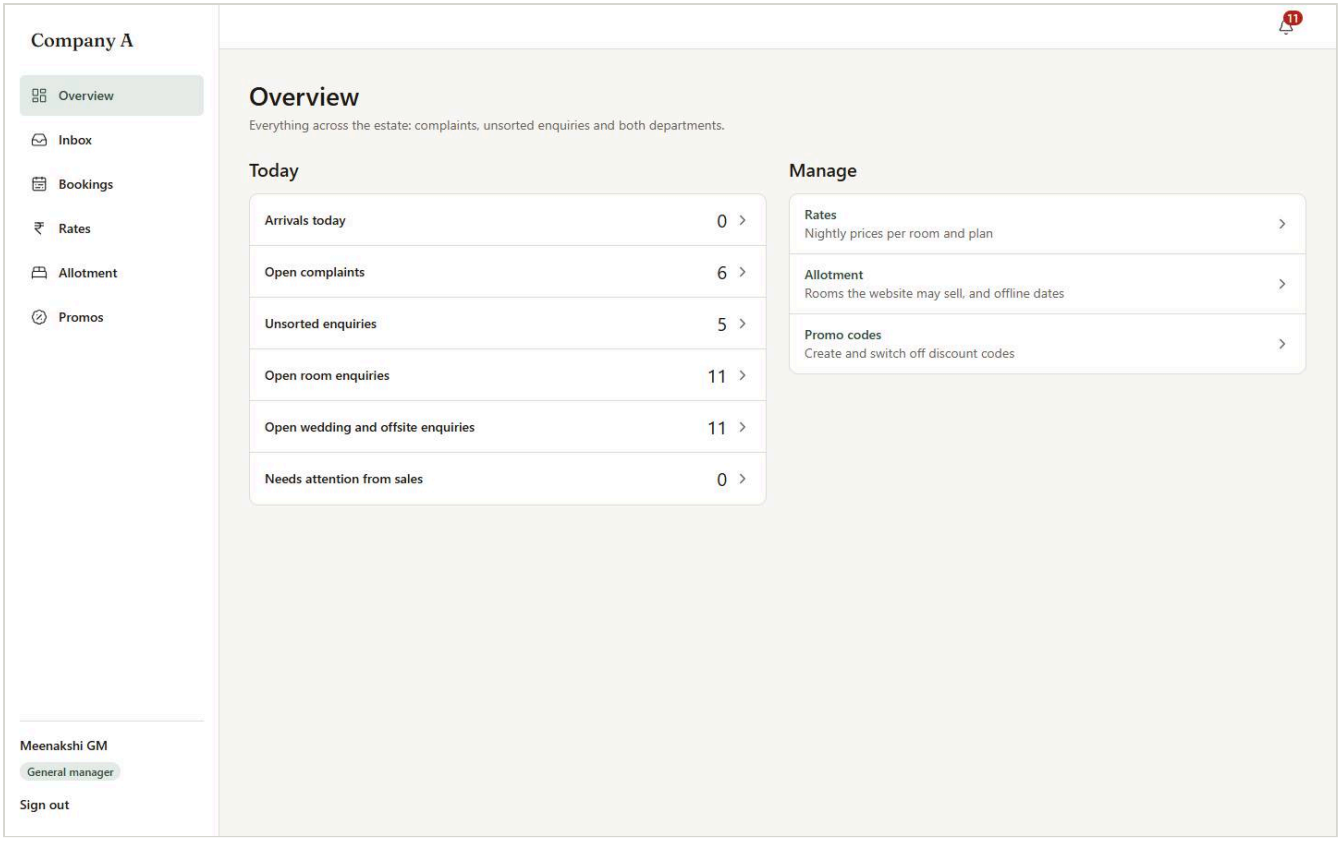


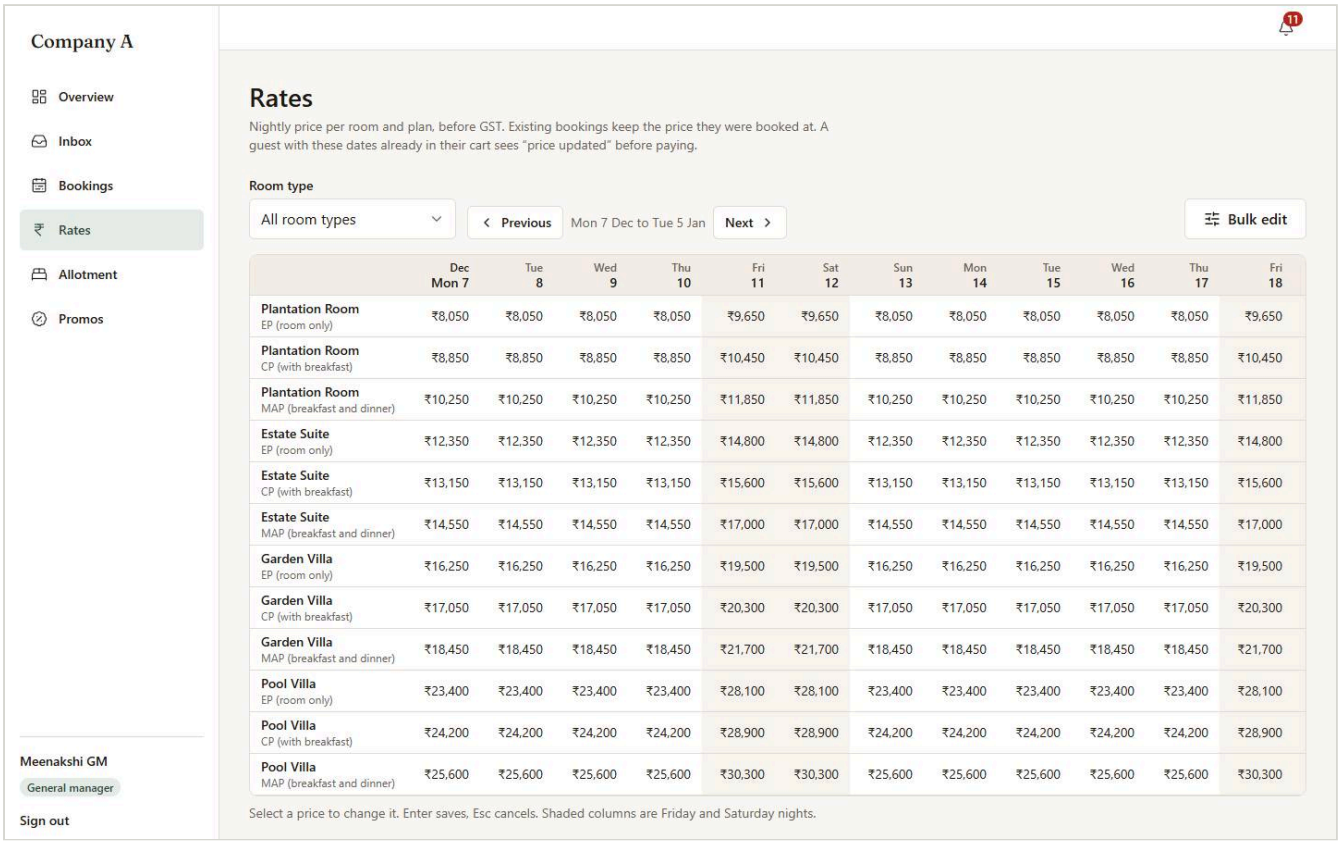
Confirmation. Large booking reference, per room GST breakdown, calendar file and email view, and the Form C note for foreign guests.



Staff screens

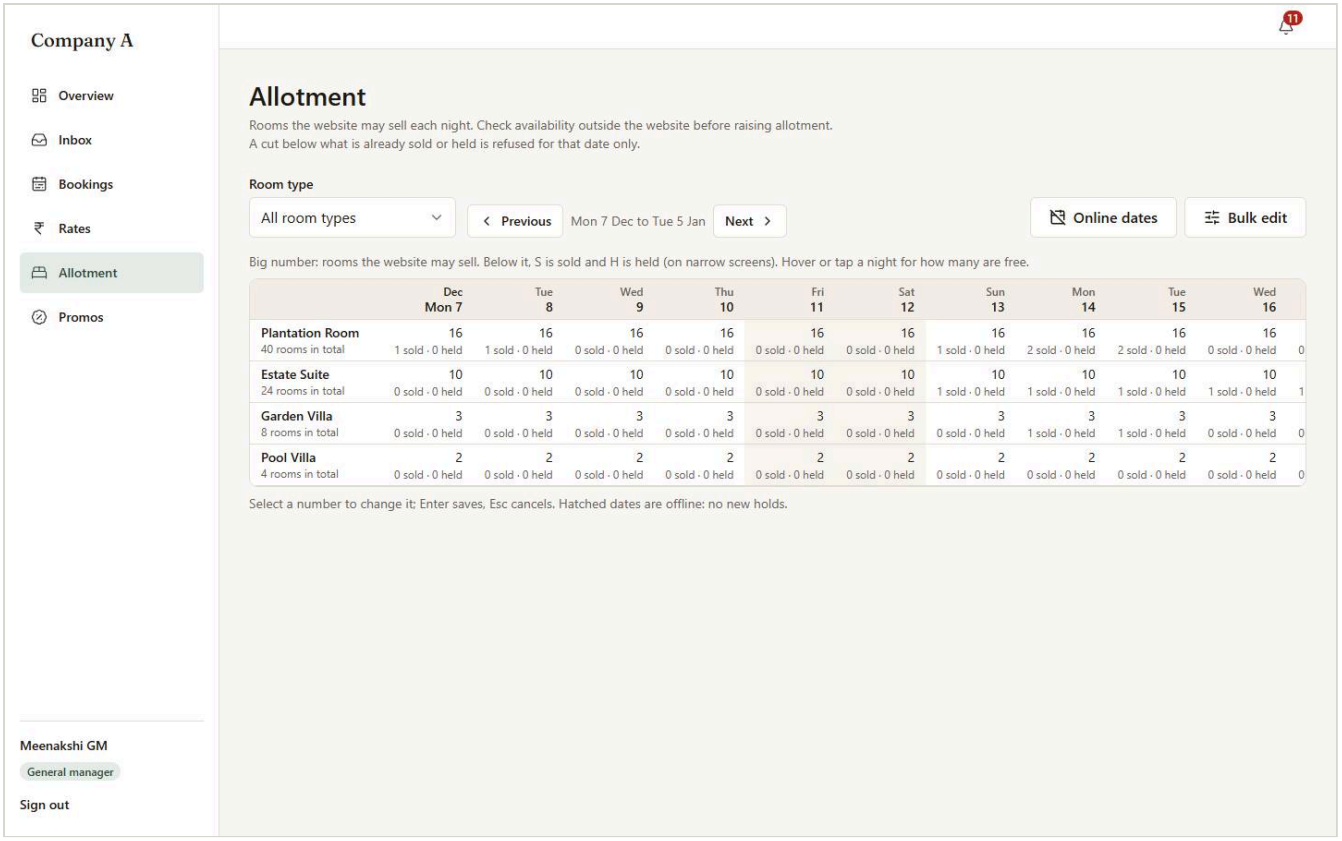
Staff screens are denser: tables with sticky headers, tabular numbers and inline edits, because staff work in them all day.



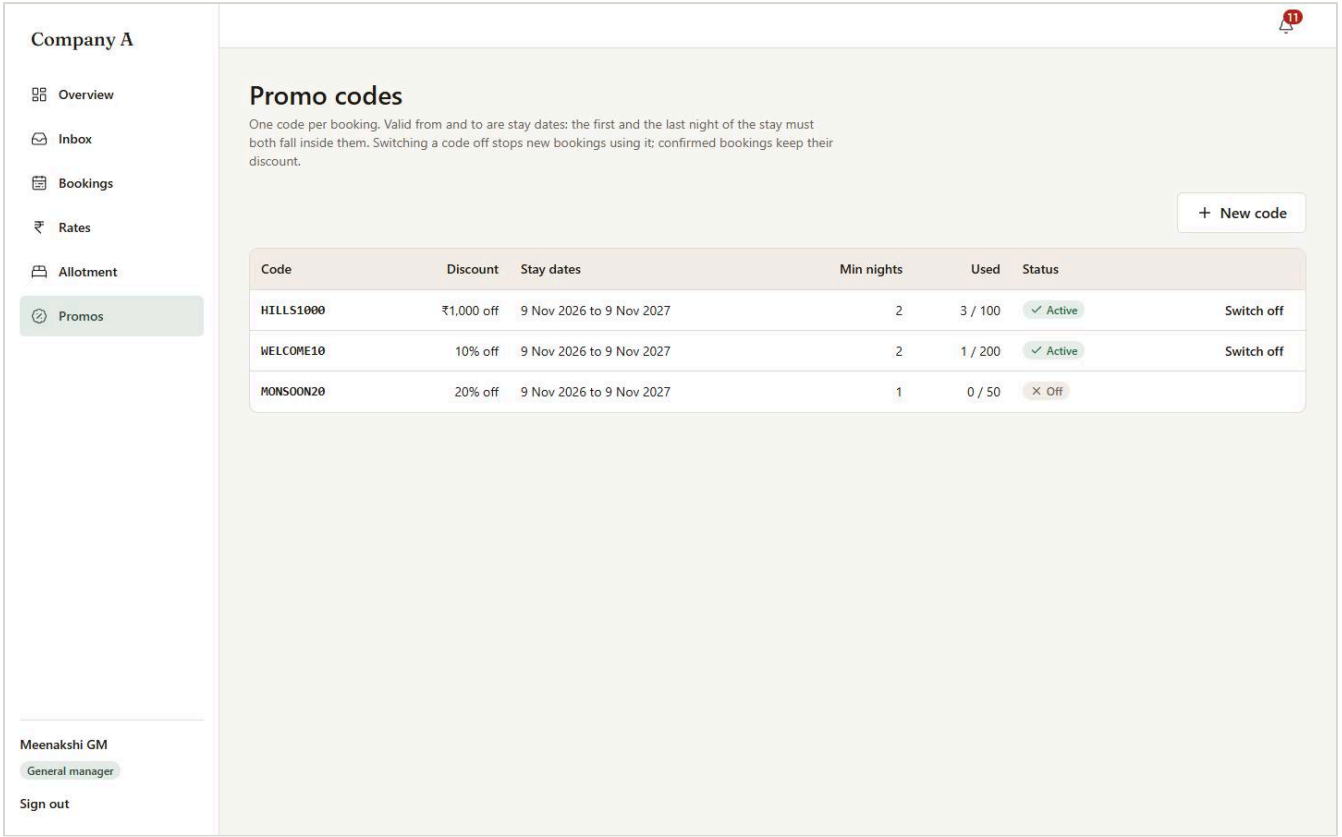


	Dec Mon 7	Tue 8	Wed 9	Thu 10	Fri 11	Sat 12	Sun 13	Mon 14	Tue 15	Wed 16	Thu 17	Fri 18
Plantation Room EP (room only)	₹8,050	₹8,050	₹8,050	₹8,050	₹9,650	₹9,650	₹8,050	₹8,050	₹8,050	₹8,050	₹8,050	₹9,650
Plantation Room CP (with breakfast)	₹8,850	₹8,850	₹8,850	₹8,850	₹10,450	₹10,450	₹8,850	₹8,850	₹8,850	₹8,850	₹8,850	₹10,450
Plantation Room MAP (breakfast and dinner)	₹10,250	₹10,250	₹10,250	₹10,250	₹11,850	₹11,850	₹10,250	₹10,250	₹10,250	₹10,250	₹10,250	₹11,850
Estate Suite EP (room only)	₹12,350	₹12,350	₹12,350	₹12,350	₹14,800	₹14,800	₹12,350	₹12,350	₹12,350	₹12,350	₹12,350	₹14,800
Estate Suite CP (with breakfast)	₹13,150	₹13,150	₹13,150	₹13,150	₹15,600	₹15,600	₹13,150	₹13,150	₹13,150	₹13,150	₹13,150	₹15,600
Estate Suite MAP (breakfast and dinner)	₹14,550	₹14,550	₹14,550	₹14,550	₹17,000	₹17,000	₹14,550	₹14,550	₹14,550	₹14,550	₹14,550	₹17,000
Garden Villa EP (room only)	₹16,250	₹16,250	₹16,250	₹16,250	₹19,500	₹19,500	₹16,250	₹16,250	₹16,250	₹16,250	₹16,250	₹19,500
Garden Villa CP (with breakfast)	₹17,050	₹17,050	₹17,050	₹17,050	₹20,300	₹20,300	₹17,050	₹17,050	₹17,050	₹17,050	₹17,050	₹20,300
Garden Villa MAP (breakfast and dinner)	₹18,450	₹18,450	₹18,450	₹18,450	₹21,700	₹21,700	₹18,450	₹18,450	₹18,450	₹18,450	₹18,450	₹21,700
Pool Villa EP (room only)	₹23,400	₹23,400	₹23,400	₹23,400	₹28,100	₹28,100	₹23,400	₹23,400	₹23,400	₹23,400	₹23,400	₹28,100
Pool Villa CP (with breakfast)	₹24,200	₹24,200	₹24,200	₹24,200	₹28,900	₹28,900	₹24,200	₹24,200	₹24,200	₹24,200	₹24,200	₹28,900
Pool Villa MAP (breakfast and dinner)	₹25,600	₹25,600	₹25,600	₹25,600	₹30,300	₹30,300	₹25,600	₹25,600	₹25,600	₹25,600	₹25,600	₹30,300

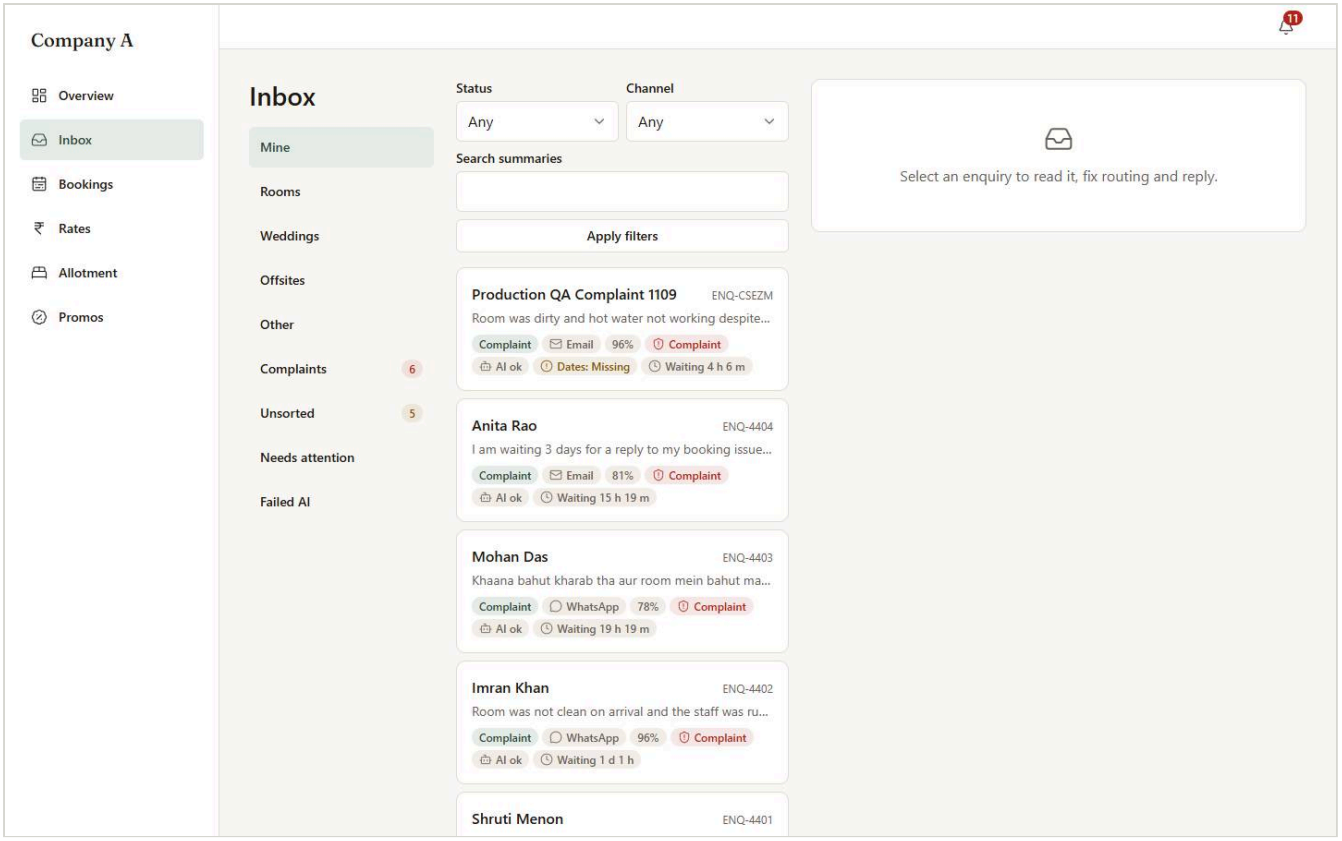
Rates. Room type by plan by date. Weekends shaded. Bulk edit shows how many prices will change before applying.



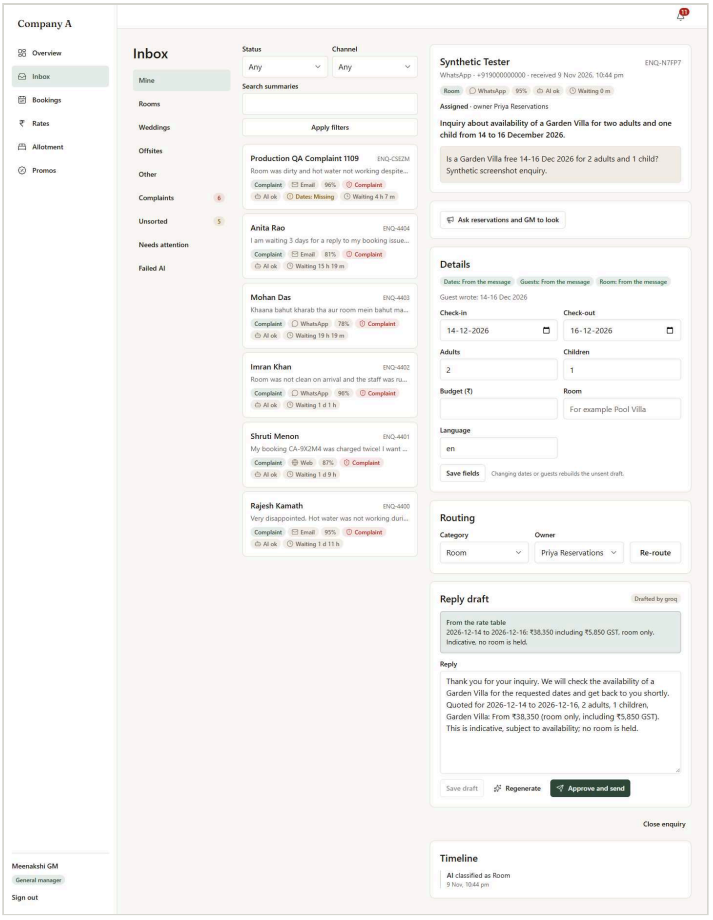
Allotment. The allotment number is large; sold and held are small beneath it. Refused edits turn red with the reason.



Promos and blackouts. Promo codes list with status labels and a create form. Blackouts are set from a panel on the Allotment page (take offline or bring online, with a required reason).

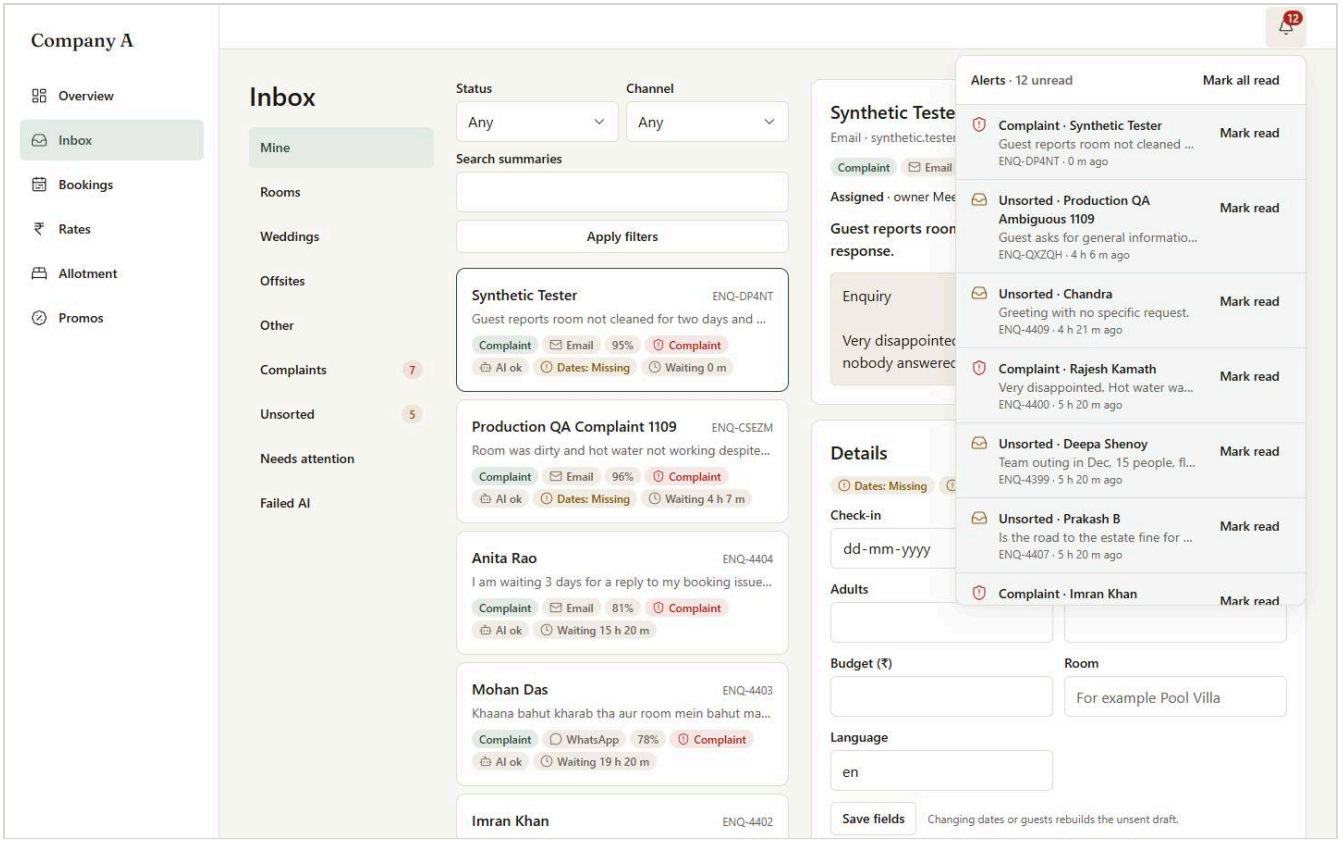


Inbox. Lanes by role (Mine, Rooms, Weddings, Offsites, Other, Complaints, Unsorted, Needs attention, Failed AI). Each row shows channel, category, a confidence badge (warning colour under 70%) and time waiting.

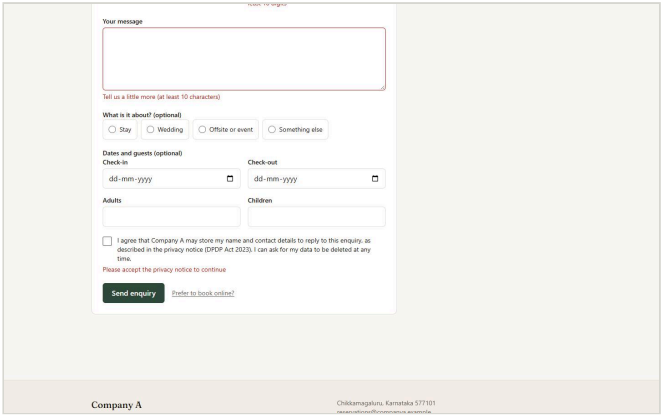


Enquiry detail. The original message, the AI category, confidence and owner, extracted fields with "From the message" or "Edited by staff" badges, the quote block labelled "From the rate table", and the draft editor. Synthetic enquiry captured on the live app on 9 November 2026.

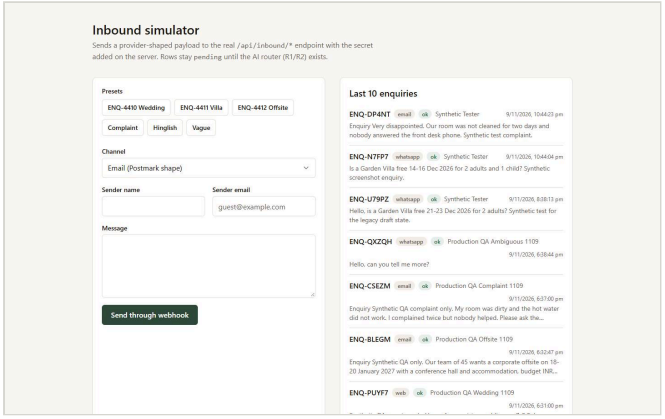
Quote out of date (pending deployment). The fix adds a warning panel ("Quote being rebuilt" or "Quote out of date") with Retry, disables Approve until the quote matches the saved details, and shows the dates, party and room each draft was built on. It is not on the live app yet, so no screenshot is included.

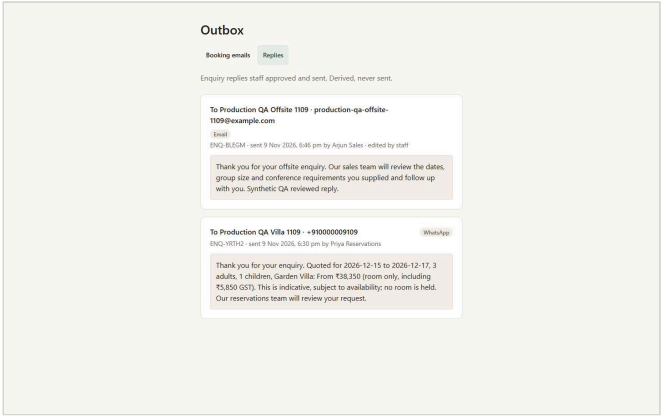






States. Lists use a shared empty state with a next step (for example "Create the first code" on Promos), and a shared error state with Retry. Forms keep the input on error and show the message next to the field. Pages that stream show skeletons while data loads. Production QA did not force every loading and error state (section 12).





Performance on slow hill networks

Lighthouse 13.6.0 on the live site, mobile, simulated Slow 4G, median of three runs (production QA, 9 November 2026):

Page	Median LCP	CLS	Median JS transfer
`/`	2.157 s	0	169.5 KB
`/rooms`	2.230 s	0	196.1 KB
`/rooms/garden-villa`	2.458 s	0	196.3 KB
`/book/details`	2.005 s	up to 0.065	205.7 KB
`/enquire`	0.958 s	0	177.2 KB

All five medians are under the 2.5 s target. `/rooms` had two runs with long tasks (TBT 1,749 and 1,379 ms) and one LCP of 3.071 s; the cause is not established (section 12). What helped: server rendering with little client JavaScript (own JavaScript 13 to 26 KB per page), `next/image` with AVIF and WebP, a priority hero at quality 60, lazy images below the fold, and the Vercel function region matched to the database.

---

9. Security and compliance

Area	What is in place
Role based access	One role by capability table. Pages render a 403 screen, actions return FORBIDDEN, and every admin service checks again. The inbox filters lanes by role in SQL
Ownership	A guest reads only their own bookings (`requireGuestOwner`). The guest id always comes from the session, never the request
Input validation	Zod on every server action, route handler, webhook and AI answer
Machine endpoints	Cron needs a Bearer `CRON_SECRET`. Inbound webhooks need `x-inbound-secret`. The mock payment webhook needs an HMAC SHA-256 signature. All fail closed
Secrets	None in the repository. `.env*` files are git ignored; only `.env.example` with names is tracked. A scan of tracked files on 9 November 2026 found no keys or connection strings (only the local Docker URL and fake test URLs)
Headers	HSTS two years, `X-Frame-Options: DENY`, nosniff, Referrer-Policy, Permissions-Policy, a CSP
Cookies	Secure, HttpOnly, SameSite=Lax on the live site (production QA)
Rate limits	Sign in, sign up, enquiry form, hold and pay, per IP and per email, contact or guest
Logging	Errors logged as class name and SQL state only, never values
DPDP Act 2023	Consent checkbox with version and time stored at sign up and on enquiries. Only name, email and phone are collected. Contact details are kept out of the message text. Passports are never taken online (Form C at the desk). No real personal data anywhere

Demo limits, stated plainly.

1. Demo staff logins are public, so anyone can sign in as staff on the live demo. Acceptable only because all data is synthetic.

2. Rate limits are in memory per server instance. A distributed attacker could exceed them. A shared store (Postgres or Redis) is the production step.
3. The CSP allows inline scripts (`'unsafe-inline'`) to keep partial prerendering working.
4. Denied pages render a 403 or 404 screen with HTTP status 200 because of streaming.
5. `next-auth` is a beta release (5.0.0-beta.32).

---

10. Testing and quality

Test status on 9 November 2026.

Suite	Result	Notes
Unit (Vitest)	752 of 752 pass (735 before today)	No database
PostgreSQL quote lifecycle suite (new, `inbox-quote.test.ts`)	16 of 16 pass	QA-01, QA-02 and failed rebuild regressions
PostgreSQL inbox suite (`inbox.test.ts`)	18 of 18 pass	
Full PostgreSQL integration suite	Did not complete today (local memory limit)	Last full run: 176 of 176 pass
Typecheck, lint, production build	Pass	`pnpm typecheck && pnpm lint && pnpm build`
Playwright end to end	6/6 at the last run (Phase 6, against a local build)	Not rerun today; today's browser checks ran on the live site instead (below)

What the tests cover. The nine mandatory booking invariants (section 6b), multi room races and deadlock checks, hold idempotency, admin editors racing holds, GST boundaries, AI claim and recovery with `SKIP LOCKED`, the three attempt cap, complaint alerts, grounding, the price guard, PII redaction, provider failover, and since today the quote lifecycle: named villa kept on date and guest edits, staff room beats the message, room only edit cannot be approved from a stale tab, a forced rebuild failure and an in flight rebuild refuse approval, and a send racing an edit matches the facts at send time.

CI. GitHub Actions on every push to `main` and every pull request: typecheck, lint, format check, unit tests and build; then the integration suite against a disposable Postgres 16 service.

Production QA (9 November 2026, independent audit on the live site). 22 screen templates and 34 named browser scenarios passed. Five findings: QA-01 and QA-02 (P1, quote integrity), QA-05 (P1, bulk editor lost on a network failure), QA-03 (P2, date toolbar overflow at 320 px), QA-04 (P2, unsaved draft text lost when a stale send is refused). No P0 booking, payment, inventory or data exposure defect was observed.

Fixed after QA (status on 9 November 2026).

Finding	Fix	Status
QA-01 date edit loses named villa	Room keeps the guest's preference; three room states	Fixed and tested on `claude/s3-tdd-pack`, pending deployment
QA-02 obsolete quote approvable	Every quote affecting edit rebuilds; approval checks saved facts under the enquiry lock	Fixed and tested on `claude/s3-tdd-pack`, pending deployment

Finding	Fix	Status
Failed rebuild risk	Failed or in flight rebuild blocks approval; Retry	Fixed and tested on `claude/s3-tdd-pack`, pending deployment
QA-05 bulk editor lost on network failure	Inputs kept, inline error with Retry, never reports success	Fixed on a local branch, pending review and deployment
QA-03 date toolbar overflow at 320 px	Toolbar stacks under the small breakpoint	Fixed on a local branch, pending review and deployment
QA-04 unsaved prose lost on stale refusal	Unsaved text kept under the current draft; refusal unchanged	Fixed on a local branch, pending review and deployment

11. What was cut and why

Item	Why
A6 GST slab editor (P2)	Slabs change rarely and are seeded; the time went to allotment safety
A7 daily booking export (P2)	Optional in the brief. The `export_runs` table exists but no job uses it
Q8 first reply metrics screen (P1)	`first_reply_at` is stored, but the screen and working hours definition were not built
Real payment gateway	Out of scope; a mock with a signed webhook exercises the same exactly once path
Real email and WhatsApp	Simulated with real payload shapes; Meta signature verification and the GET handshake are not implemented
Channel manager or reservation system integration	Out of scope in the brief
Deploying the QA fixes before the deadline	The full PostgreSQL run needed before merging did not complete on the local machine (memory), so the fixes were not merged to `main`. They are on branches, tested as listed in section 10

12. Known limitations

1. **Deployment configuration not verified from the dashboard.** Region `cle1`, the daily cron and the Neon region (`us-east-2`) are set in the repository; we had no Vercel dashboard access to confirm the cron is active or which database branch production uses.
2. **AI draft prose quality.** The safe template is used when model prose fails the guard (7 of 30 on 120b). It is correct but generic and can ask for facts the guest already gave. A human reviews every draft.
3. **`/rooms` long tasks.** Two of three live Lighthouse runs showed 1.4 to 1.7 s of blocking time and one 3.07 s LCP. Cause not established.
4. **Rate limits in memory**, inline script CSP and public demo logins (section 9).
5. **PII redaction** is pattern based.
6. **Promo `max_uses`** can be exceeded by one or two under heavy concurrency (accepted).
7. **Expiry sweep** is one statement over all expired holds, not batched.
8. **Missing indexes** on `payments.booking_id`, `booking_events.booking_id`, `enquiry_events.enquiry_id`, and enquiry search uses `ILIKE` on JSON. Fine at demo size.

9. **Webhook dedupe** uses an advisory lock and a scan, not a unique index.
10. **Zero total stays** cannot be paid online. Failed and timed out payment attempts are not stored.
11. **Verified only locally, not on the live site**: concurrent holds, late payment webhooks, real shared admin writes, provider failover under a 429 storm, and the stale AI recovery cap. They are covered by the PostgreSQL suite, which does not run against production.
12. **QA fixes not live.** On the live app, a date only edit can still re-quote a cheaper room (QA-01) and a room only edit can still leave an old price approvable (QA-02); staff must check the quote against the saved details before approving. The fixes are on `claude/s3-tdd-pack`. QA-05, QA-03 and QA-04 are fixed on local branches only.
13. **Full PostgreSQL run not repeated today.** The last complete run passed 176 of 176; today only the quote (16) and inbox (18) suites were confirmed alongside the unit tests.
14. **Not every loading and error state** was forced during production QA.

---

13. What I would do next

Priority	Work	Why (metric)
0	Complete the full PostgreSQL run, merge `claude/s3-tdd-pack`, deploy, then review and deploy the QA-05, QA-03 and QA-04 branches, and recheck each on the live app	Correct quotes are a precondition for fast first replies; staff tools must not lose work
1	Q8 response metrics: median first reply in working hours, per lane and owner	Measures the 30 minute target directly; the data is already stored
2	Real WhatsApp Business and email integration with signature checks	Most enquiries arrive there today; required before the inbox replaces the GM's phone
3	Real payment gateway (Razorpay or similar) using the same webhook path	Required to take real direct bookings (direct share 12% to 20%)
4	Shared rate limit store, CSP nonces, change demo passwords	Production security
5	A7 daily booking export to the reservation system	Removes double entry and keeps the reservation system as the source of truth
6	Field performance data (CrUX) and fixing `/rooms` long tasks	Conversion on slow hill networks
7	Missing indexes and batched expiry	Scale beyond demo traffic
8	Draft prose quality: measure the template rate weekly and tune the prompt	Faster first replies when staff edit less

---

14. Appendix

Environment variables (names only)

Name	Purpose
`DATABASE_URL`	Pooled Postgres connection used by the app
`DATABASE_URL_UNPOOLED`	Direct connection for migrations and seed scripts
`AUTH_SECRET`	Auth.js signing secret

Name	Purpose
`CRON_SECRET`	Bearer token for the cron route
`PAYMENT_WEBHOOK_SECRET`	HMAC key for the mock payment webhook
`INBOUND_WEBHOOK_SECRET`	Shared secret for the inbound email and WhatsApp webhooks
`AI_PROVIDER`	`groq`, `google`, `anthropic`, `local` or `rules`
`AI_MODEL`	Model id, for example `openai/gpt-oss-120b`
`GROQ_API_KEY_1` to `GROQ_API_KEY_4`	Groq keys, tried in order
`GOOGLE_GENERATIVE_AI_API_KEY`, `ANTHROPIC_API_KEY`, `LOCAL_AI_BASE_URL`	Other providers
`HOLD_MINUTES`	Hold length, default 10
`DEV_TOOLS_ENABLED`	Opens `/dev` (simulator, outbox)
`TEST_DATABASE_URL`, `DB_PROTECTED_HOSTS`	Local only: integration test database and hosts the destructive scripts refuse

Routes

Area	Routes
Guest	`/`, `/rooms`, `/rooms/[slug]`, `/book/details`, `/book/pay/[ref]`, `/booking/[ref]`, `/booking/[ref]/email`, `/booking/[ref]/ics`, `/account`, `/enquire`, `/login`, `/signup`
Staff	`/admin/login`, `/admin`, `/admin/rates`, `/admin/allotment`, `/admin/promos`, `/admin/bookings`, `/admin/bookings/[ref]`, `/admin/inbox`, `/admin/inbox/[id]`
Machine	`/api/cron/expire-holds`, `/api/inbound/email`, `/api/inbound/whatsapp`, `/api/payments/mock`, `/api/auth/[...nextauth]`
Dev tools	`/dev`, `/dev/test`, `/dev/outbox`, `/dev/login`, `/dev/ui`

Glossary

Term	Meaning
Allotment	Rooms of a type per night that the website may sell
Hold	A 10 minute reservation of stock while the guest pays
EP, CP, MAP	Room only; with breakfast; with breakfast and one main meal
GST slab	The GST rate band chosen by the nightly room value
Paise	One hundredth of a rupee. All money is stored as whole paise
Unsorted lane	Enquiries the AI could not classify with confidence. The GM sees them
Grounding	Server check that extracted dates and counts appear in the original message
Quote basis	The saved facts a draft's price was built from
DPDP Act	India's Digital Personal Data Protection Act, 2023
Form C	The foreigner registration form hotels file at check in